

Gnomore Gnomes

Documento de diseño — juego de estrategia por turnos inspirado en Polytopia

Generado a partir de la conversación de brainstorming con Jesús · 13 de septiembre de 2026 · actualizado 17 de septiembre de 2026

Estado: diseño en curso — alcance del MVP, estilo visual, formato de trabajo, nombre y primera raza ya decididos. Escenario, cámara y un primer sistema de unidades (movimiento y combate cuerpo a cuerpo) ya implementados y probados en modo sandbox — la mecánica real de partida (turnos, cómo se gana/pierde) todavía está sin definir.

1. Resumen del proyecto

Juego web de estrategia por turnos, construido igual que Goal2Goat (a través de Claude, sin que Jesús escriba código directamente). Se inspira en Polytopia (mapa, razas, árbol de progresión, unidades) pero no lo copia: añade un concepto propio centrado en la caza de gnomos como "deporte" entre razas.

El proyecto está pensado desde el día uno para ser multiplataforma (navegador, APK móvil y Steam) con una única partida guardada en la nube que sigue al jugador esté donde esté, con multijugador, idiomas seleccionables, progresión de perfil y compras dentro del juego.

2. Decisiones ya cerradas

Nombre del juego	Gnomore Gnomes
Alcance del MVP	Mínimo: mapa pequeño, progresión corta, 1 rival IA básico, sin multijugador ni backend todavía — empezando con una sola raza jugable (Mushboom Forest). Objetivo: validar que el "deporte del gnomo" es divertido antes de ampliar.
Estilo visual	Minimalista/geométrico, como el propio Polytopia: formas planas/hexágonos, colores sólidos, iconos simples.
Formato de trabajo	Igual que Goal2Goat: carpeta de proyecto propia, cada entrega es un zip acumulativo con TODOS los archivos del juego; Jesús prueba en local y da feedback con capturas.

3. Concepto y narrativa

Un mundo de fantasía habitado por las razas clásicas del género — orcos, elfos, enanos, humanos — junto a algunas razas nuevas/originales, sufre una invasión de gnomos. A raíz de esta invasión surge un "deporte" entre las razas: competir para ver cuál elimina más gnomos.

El mapa está lleno de gnomos, que son escurridizos y se mueven activamente para evitar ser atrapados. La misión central del jugador es cazarlos.

Razas

El MVP arranca con una sola raza jugable, pero el sistema de razas debe diseñarse desde el principio para que añadir nuevas sea sencillo (ver regla de oro de escalabilidad, sección 10).

Mushboom Forest (primera raza, MVP)

Orcos, goblins y similares que habitan el bosque Mushboom. Una de sus habilidades características es cultivar setas bomba.

4. Mecánicas principales

Puntos de acción

Los jugadores ganan puntos de acción que determinan cuánto pueden hacer en su turno; el turno termina cuando se agotan. Formas de conseguir puntos de acción:

- Tener un gnomo capturado / en tu posesión al final del turno.
- Golpear a un gnomo mientras está capturado/sujeto.

Al golpear a un gnomo capturado, este se va "cargando": cuanto más se le golpea, más puntos o valor acumula.

Ataque a bases y condición de eliminación

El objetivo estratégico es llegar a las bases rivales (ciudades/pueblos), atacarlas para romper sus defensas, y entrar con un gnomo capturado para "machacarlo" dentro de la base enemiga — esto destruye los edificios del rival. Un jugador que se queda sin edificios queda eliminado de esa partida.

Condición de victoria

La partida termina cuando un jugador ha acumulado más "puntos de victoria" que el resto — un tipo de puntuación distinto de los puntos de acción, que se gana según cómo se juegue el deporte a lo largo de la partida. Ese jugador es el ganador.

Unidades y bases

Los jugadores pueden crear unidades con características distintas desde sus bases, y mejorar tanto sus bases como sus defensas.

Nota de diseño: muchas mecánicas estarán INSPIRADAS en Polytopia, explícitamente sin copiarlas.

5. Progresión

Árbol de habilidades dentro de la partida

Durante una partida, los jugadores desbloquean nuevas habilidades y nuevas unidades utilizables solo para esa partida (progresión temporal, reseteable cada vez).

Progresión de perfil persistente

Por separado, existe una progresión de perfil de jugador que persiste entre partidas: logros, habilidades, etc., siguiendo el mismo enfoque que se implementó en Goal2Goat.

6. Multiplataforma, multijugador y guardado

Plataformas objetivo

- Navegador web (versión base).
- APK para móvil.
- Steam (versión de escritorio) — a futuro.

Multijugador

El juego debe soportar multijugador. El enfoque planteado es por turnos asíncrono (como el multijugador real de Polytopia): cada jugador juega su turno cuando puede, y el rival recibe la partida actualizada para jugar el suyo — evita la complejidad de sincronizar partidas en tiempo real.

Guardado: partida local + progreso en la nube

Decisión (actualizada el 14/09/2026): se separan dos tipos de datos con necesidades distintas.

- **Partida en curso:** se guarda en local, en el dispositivo/navegador donde se juega — igual que el Polytopia real. Cerrar la pestaña o la app a medias NO la pierde: queda guardada ahí y se puede retomar más tarde en ese mismo dispositivo. Lo único que no hace es viajar a otro dispositivo — si se abre el juego desde otro sitio, esa partida concreta no aparece.
- **Progreso de jugador:** perfil, logros, desbloques y todo lo "conseguido" se guarda en la base de datos de Firebase asociada a la cuenta del jugador, y ese sí viaja con él a cualquier dispositivo.

Esto sustituye la idea inicial de tener el estado completo de la partida sincronizado en la nube entre dispositivos: solo el progreso de perfil se sincroniza así, lo cual simplifica bastante la arquitectura (no hace falta sincronizar el tablero turno a turno contra un servidor).

7. Monetización

Se contempla la venta de contenido dentro del juego, como nuevas razas o skins.

8. Idiomas

El juego debe permitir elegir idioma. Este requisito se integra además como regla de oro (ver sección 9): todo el texto del juego debe construirse desde el principio pensando en traducción a varios idiomas, en vez de añadirse más adelante.

9. Infraestructura técnica prevista

- **Hosting inicial:** GitHub Pages.
- **Hosting futuro:** migración de pago a Cloudflare una vez el juego funcione bien.
- **Base de datos:** Firebase, plan gratuito.

10. Reglas de oro (a tener en cuenta SIEMPRE al implementar)

Estas reglas se aplican a todo el código que se genere para este proyecto. Jesús indicó explícitamente que esta lista puede ampliarse o modificarse con el tiempo — se mantiene aquí como la versión vigente en la fecha de este documento.

1. Un archivo por mecánica

Cada mecánica de juego debe implementarse en un archivo distinto, para facilitar su futura mejora o corrección, y de forma que ninguna mecánica necesite conocer el funcionamiento interno de otra. Implementación concreta adoptada para las unidades (ver sección 11.1): un archivo "núcleo" (js/units.js) que sabe crear, pintar, seleccionar y desplazar unidades, y al que cada mecánica de acción (movimiento, combate, y las que se añadan después) se conecta registrándose como "proveedor de rango" — así se puede reescribir o ampliar una mecánica sin tocar las demás ni el núcleo.

2. Preparado para varios idiomas desde el inicio

Todo debe construirse teniendo en cuenta que el juego estará traducido a varios idiomas.

3. Fácil de exportar a APK y Steam

Todo debe crearse y programarse pensando en que sea muy fácil exportarlo a una APK y, en el futuro, a un juego de Steam.

4. Fácil de portar a cualquier sistema

El desarrollo debe ser muy sencillo de portear a cualquier sistema, más allá de los tres anteriores.

5. Interfaz adaptada a cada modo de uso

El juego debe adaptar su interfaz a los distintos modos (escritorio, móvil...), facilitando una navegación sencilla, intuitiva y cómoda en cada uno — mismo enfoque que las interfaces diferenciadas de escritorio y móvil ya usadas en Goal2Goat.

6. Controles siempre cómodos e intuitivos

Los controles del juego deben ser siempre muy cómodos e intuitivos para el jugador.

7. Retroalimentación constante al jugador

Todo lo que ocurra en el juego debe generar retroalimentación: sonidos y pequeñas animaciones de sprites y/o menús. Ejemplos citados: pasar el ratón por encima de un menú lo resalta de forma vistosa y agradable; atacar a un enemigo en el tablero provoca una pequeña animación de pulso en el sprite enemigo junto con un sonido.

8. Generar adicción y dopamina en el jugador

Marcada explícitamente por Jesús como una de las reglas de oro más importantes: todo lo que se cree debe estar orientado a generar adicción y dopamina en el usuario.

9. Escalabilidad fácil en todas las mecánicas

El juego debe tener una escalabilidad muy fácil y sencilla de implementar en todas sus mecánicas — por ejemplo, añadir una nueva raza (más allá de Mushboom Forest) debe ser sencillo gracias a este principio.

10. Soporte de Gamepad

Todo debe programarse teniendo en cuenta que el usuario puede decidir en cualquier momento manejar la navegación de menús, interfaces y el propio juego con un mando (Gamepad) — incluso jugando desde la web, detectándolo vía la Gamepad API del navegador.

11. Uso de iconos Phosphor

Se pueden y se deben utilizar iconos Phosphor en toda la interfaz para que el juego sea visualmente más atractivo — mismo criterio ya usado en Goal2Goat.

12. Nivel Triple A en todo

Toda decisión de diseño y programación — optimización, gráficos, y cualquier otro aspecto — debe tomarse como si se tratara de un juego triple A. Marcada explícitamente como una regla importante.

11. Estado de implementación (a fecha de este documento)

- **Infraestructura:** repositorio en GitHub (tiopops/GnomoreGnomes) y GitHub Pages activo en <https://tiopops.github.io/GnomoreGnomes/>.
- **Flujo de entrega:** script "subir.bat" de doble clic que conecta/repara la carpeta local con el repositorio y hace commit + push automáticamente (mensaje = fecha y hora).
- **Menú principal:** implementado — Nueva Partida, Reanudar Partida (activo solo si hay partida guardada) y Multijugador (desactivado a propósito, sin backend todavía).
- **Flujo de Nueva Partida:** implementado — elegir modo (solo GnomeSmash por ahora) → elegir raza (solo El Reino Mushboom Forest por ahora) → elegir número de rivales (solo 1 por ahora) → generación del escenario.
- **Escenario:** tablero de losetas de hierba en perspectiva isométrica (estilo Polytopia), tamaño 13×13 para 1 rival. Losetas generadas por código como placeholder — Jesús proporcionará las losetas de arte definitivas más adelante para sustituirlas sin tocar el resto del motor.
- **Cámara:** paneo y zoom con inercia, con el zoom anclado al puntero del ratón (el punto del tablero bajo el cursor se queda fijo al hacer zoom) y un rango de zoom limitado para que las losetas nunca se vean borrosas por ampliación.
- **Unidades — movimiento y combate:** primer sistema jugable de unidades sobre el tablero, con una unidad de prueba del jugador (Mushboom Forest) y un rival de prueba colocado en una casilla al azar. Implementado: selección con resaltado, radio de movimiento y desplazamiento paso a paso con animación de salto y giro hacia la dirección real; ataque cuerpo a cuerpo (el atacante debe terminar a 1 casilla del rival — si el movimiento se lo permite, se acerca por el camino más corto y golpea en el mismo clic); vida (3 puntos por unidad de momento), barra de vida visible al seleccionar una unidad o al tener un rival dentro de rango de acción, texto de daño flotante, y eliminación de la unidad al llegar a 0 de vida. Ver sección 11.1 para cómo está organizado el código.
- **Retroalimentación:** sonidos generados por código (sin archivos de audio todavía) para pasar el ratón, pulsar en menús/tarjetas, cada paso de movimiento, golpear y eliminar a un rival; animación sutil de "respiración" en las unidades quietas.
- **Pendiente:** la mecánica real de partida (turnos, cómo se gana o se pierde, puntos de acción, captura de gnomos, ataque a bases) — de momento el movimiento y el combate se están probando en un tablero de pruebas ("sandbox"), sin reglas de turno ni condición de victoria todavía.

11.1 Arquitectura del sistema de unidades

Movimiento y combate se están probando y ampliando muy rápido, así que este sistema se reorganizó explícitamente para poder seguir añadiendo o modificando mecánicas sin arriesgarse a romper las que ya funcionan (aplicando la regla de oro 1, ver sección 10) — algo que además va a hacer falta pronto para el propio motor de partida y para mapas de losetas más complejos (obstáculos, terrenos distintos, más de un rival).

El código de unidades se divide en tres archivos con una única vía de comunicación entre ellos:

- **js/units.js (núcleo):** el único que sabe crear, pintar, seleccionar y desplazar una unidad de una loseta a otra, y ofrece utilidades compartidas (camino paso a paso, barra de vida, texto flotante, marcadores sobre el tablero, eliminar una unidad). No conoce ninguna regla de movimiento ni de combate.
- **js/movement.js y js/combat.js (mecánicas):** cada uno en su propio archivo, calculan su propio radio de acción y lo pintan sobre el tablero usando solo las utilidades del núcleo — sin saber nada el uno del otro.
- **Registro de "proveedores de rango":** al cargarse, movement.js y combat.js se registran ante units.js (Units.registerRangeProvider). Cuando el jugador selecciona una unidad, o cuando termina de moverse o atacar, units.js simplemente pide a cada mecánica registrada que vuelva a pintar su propio radio — así añadir una mecánica nueva (construir, curar, una habilidad especial...) es crear un archivo más y registrarlo, sin tocar units.js ni las mecánicas existentes.

La mecánica de partida en sí (turnos, cómo se gana o se pierde una partida — todavía sin definir, ver sección 12) está pensada para poder construirse sobre esta misma base sin reestructurar el sistema de unidades otra vez.

12. Próximos pasos

El diseño inicial ya está cerrado (ver sección 2): nombre, alcance del MVP, estilo visual, formato de trabajo y primera raza están decididos; el menú, la generación de escenario y un primer sistema de movimiento/combate ya están implementados y probados en modo sandbox (ver sección 11).

Decisiones que quedan abiertas para más adelante:

- **Mecánica de partida (todavía sin definir):** cómo se gana y se pierde una partida, si hay turnos y cómo se alternan, puntos de acción, y cómo encajan en esto la captura/carga de gnomos y el ataque a bases descritos en la sección 4 — de momento movimiento y combate solo se han probado sueltos, sin ninguna de estas reglas todavía.
- Diseño detallado de la raza Mushboom Forest: unidades concretas más allá de la de pruebas actual, estadísticas, y el resto de habilidades además de las setas bomba.
- Diseño de las próximas razas a añadir tras el MVP.
- Detalle del árbol de progresión in-game y de la progresión de perfil persistente (qué se desbloquea exactamente).
- Mapas de losetas más complejos: los actuales son una cuadrícula simple sin obstáculos ni distintos tipos de terreno — el sistema de movimiento/combate (sección 11.1) está pensado para poder crecer hacia esto sin reescribirse.
- IA de los rivales: de momento el "rival" es solo una unidad de pruebas colocada al azar, sin ningún comportamiento propio.
- Losetas de arte definitivas (las actuales son un placeholder generado por código).

Recomendación de enfoque (no es un requisito indicado por Jesús, sino una nota de planificación): dado que multijugador, cuentas con guardado en la nube, portado a APK/Steam y compras dentro del juego multiplican la complejidad entre sí, conviene validar primero el núcleo jugable en single-player con la raza Mushboom Forest — aplicando desde ya las reglas de oro de arquitectura (archivos por mecánica, i18n, portabilidad, escalabilidad) — antes de construir la capa de cuentas/backend que habilita multijugador, guardado unificado y monetización.